TRABAJO PRÁCTICO: INFORME TÉCNICO ACADÉMICO

Sistema de Gestión y Catálogo Web "Vida Natural"

1. CARÁTULA Y FICHA TÉCNICA

Campo	Detalle Académico / Institucional
Institución Educativa	Instituto Tecnológico Beltrán (ITB)
Carrera	Tecnicatura en Análisis de Sistemas
Asignatura	Algoritmos y Estructura de Datos III
Proyecto	Vida Natural — Plataforma Web de Gestión y Venta de Alimentos Saludables
Versión del Proyecto	1.0.0 (Release Académico / Producción)
Ciclo Lectivo	2026
Docente Evaluador	Saldivar Sebastian Alfredo

Ficha Técnica del Stack Tecnológico

- **Framework Principal:** Angular `v17.3.0` (Arquitectura por módulos, enrutamiento desacoplado y Lazy Loading).
- **Biblioteca de Componentes UI:** Angular Material & Angular CDK `v17.3.10` (`MatDialog`, `MatSnackBar`, `MatProgressSpinner`, `MatTooltip`, `MatButton`).
- **Lenguaje de Programación:** TypeScript `v5.4.2` (Tipado estricto, interfaces DTO, genéricos y decoradores ECMAScript).
- **Librería de Reactividad Asíncrona:** RxJS `v7.8.0` (`Observable`, `BehaviorSubject`, `Subject`, `pipe`, operadores de filtrado y transformación).
- **Estilos y Layout Responsivo:** Bootstrap `v5.3.2`, Bootstrap Icons `v1.11.3` y Vanilla CSS con Variables Custom (`styles.css`).
- **Entorno de Ejecución & Herramientas:** Node.js (`v18.x` / `v20.x LTS`) y Angular CLI `v17.3.17`.
- **Capa de Persistencia y APIs REST:**
  - **MockAPI REST API:** Almacenamiento asíncrono para productos (`/productos`) e historial de transacciones (`/compras`).
  - **Open Food Facts API:** API REST pública internacional para consulta nutricional en tiempo real.
  - **Local Storage API:** Persistencia de estado de sesión cliente.

2. ARQUITECTURA Y PATRONES DE DISEÑO

2.1 Justificación de la Arquitectura: Módulos (`NgModule`) y Lazy Loading

En el ecosistema moderno de Angular 17 conviven dos filosofías de diseño: **Standalone Components** y **Módulos tradicionales (`NgModule`)**. Para el desarrollo de *Vida Natural* se adoptó una arquitectura basada en

NgModules jerárquicos, complementada con **Lazy Loading (carga diferida)** por las siguientes razones de peso pedagógico y arquitectónico:

```
src/app/
├── app.module.ts              <-- Módulo Raíz (Eager Loading)
├── app-routing.module.ts      <-- Enrutador Principal y Protección con Guards
├── shared/
│   └── shared.module.ts       <-- Módulo Compartido (Navbar, ProductCard,
CommonModule)
└── pages/
    └── productos/
        ├── productos.module.ts         <-- Feature Module (Lazy Loading via
/productos)
        └── productos-routing.module.ts <-- Subrutas de productos
```

1. **Separación de Responsabilidades por Dominio (Domain-Driven Design):**
   - `AppModule` **(Módulo Raíz):** Encargado exclusivo de bootstrap de la aplicación, configuración de localización regional (`LOCALE_ID` para `es-AR`), importación de módulos transversales (`BrowserModule`, `HttpClientModule`, `ReactiveFormsModule`) y registro del componente base `AppComponent`.
   - `SharedModule` **(Módulo Compartido):** Centraliza componentes reutilizables sin estado de negocio global, como el `ProductCardComponent` y el `NavbarComponent`. Al exportar `CommonModule`, permite que los módulos consumidores dispongan de directivas estructurales (`*ngIf`, `*ngFor`) y Pipes sin reimportaciones redundantes.
   - `ProductosModule` **(Módulo de Funcionalidad / Feature Module):** Encapsula toda la lógica de visualización, creación, edición, borrado y confirmación de transacciones de productos.
2. **Optimización de Rendimiento mediante Lazy Loading:** En `app-routing.module.ts`, la ruta `/productos` no se compila en el paquete principal (*main bundle*), sino que se delega mediante una promesa dinámica:

   ```
   {
     path: 'productos',
     canActivate: [AuthGuard],
     loadChildren: () =>
       import('./pages/productos/productos.module').then(m =>
   m.ProductosModule)
   }
   ```

   Impacto técnico: Cuando un usuario visita únicamente el Dashboard o el Buscador Nutricional, el navegador no descarga el código JavaScript correspondiente a los formularios de creación, edición ni los diálogos modales de eliminación, reduciendo drásticamente el First Contentful Paint (FCP) y el tiempo de carga inicial.

2.2 Patrón de Inyección de Dependencias (DI) e Inversión de Control (IoC)

Angular implementa un contenedor de Inversión de Control nativo que permite el desacoplamiento total entre las clases consumidoras (componentes) y las proveedoras de lógica/datos (servicios).

```
@Injectable({
  providedIn: 'root'
})
export class ProductService {
  constructor(private http: HttpClient) {}
}
```

- **Singleton jerárquico (`providedIn: 'root'`):** Garantiza que exista una única instancia de cada servicio (`AuthService`, `ProductService`, `CompraService`, `FoodApiService`) a lo largo de todo el ciclo de vida de la aplicación.
- **Inyección por Constructor:** Los componentes reciben dependencias ya resueltas en su método constructor sin instanciarlas manualmente mediante `new`, facilitando la sustitución por *mocks* o *stubs* en pruebas unitarias con Jasmine/Karma.

2.3 Patrón MVVM / MVC en Angular

El proyecto se apega al patrón arquitectónico **Model-View-ViewModel (MVVM)**, propio de los frameworks declarativos basados en data-binding:

- **Model (Modelo de Datos):** Interfaces TypeScript puras en `src/app/models/` (`Product`, `User`, `Compra`, `AlimentoExterno`). Definen los contratos de datos y la integridad estructural en tiempo de compilación.
- **View (Vista):** Archivos de plantilla HTML (`*.component.html`) decorados con sintaxis declarativa de Angular (interpolación, bindings, directivas estructurales).
- **ViewModel (Componente / Controlador de Vista):** Clases TypeScript (`*.component.ts`) anotadas con `@Component`. Mantienen el estado reactivo, responden a interacciones del usuario y orquestan las llamadas a la capa de servicios.

2.4 Manejo de Asincronismo y Reactividad con RxJS

El proyecto adopta un paradigma de **programación reactiva funcional** a través de la librería RxJS:

1. **Gestión de Estado Reactivo con `BehaviorSubject` (`AuthService`):** A diferencia de un `Subject` estándar, el `BehaviorSubject` almacena el último valor emitido y lo envía de inmediato a cualquier nuevo suscriptor:

   ```
   private usuarioActual = new BehaviorSubject<User | null>
   (this.obtenerUsuarioGuardado());

   get usuario$(): Observable<User | null> {
     return this.usuarioActual.asObservable();
   }
   ```

Esto permite que el `NavbarComponent` reaccione instantáneamente al inicio o cierre de sesión sin requerir recargas de página.

2. **Optimización de Búsqueda con Operadores Temporales (`ProductosListaComponent`):** Para evitar peticiones redundantes o sobrecarga en el filtrado al escribir en el catálogo, se implementó un flujo reactivo con control de concurrencia:

```
this.busquedaSubject.pipe(
  debounceTime(300),          // Espera 300 ms de inactividad
  distinctUntilChanged(),     // Emite solo si el término cambió respecto al
anterior
  takeUntil(this.destroy$)    // Previene Memory Leaks al destruir el
componente
).subscribe(() => this.aplicarFiltros());
```

3. **Composición Asíncrona con `switchMap` (`ProductService`):** En operaciones compuestas (como comprar o reponer stock), se requiere consultar previamente el estado actual en el servidor y luego emitir una petición HTTP de modificación. El operador `switchMap` cancela suscripciones obsoletas y encadena observables de forma fluida:

```
comprarProducto(id: number, cantidad: number): Observable<Product> {
  return this.getProductById(id).pipe(
    switchMap(producto => {
      // Validaciones de stock y disponibilidad
      const nuevoStock = producto.stock - cantidad;
      return this.updateProduct(id, { stock: nuevoStock, disponible:
nuevoStock > 0 });
    })
  );
}
```

---

3. DOCUMENTACIÓN TÉCNICA DEL CÓDIGO FUENTE

A continuación se realiza la disección técnica exhaustiva de cada capa y componente del sistema, explicando responsabilidades, decoradores, métodos y técnicas de enlace de datos.

3.1 Modelos e Interfaces de Datos (`src/app/models/`)

A. `product.model.ts`

Contrato que representa un artículo comercial dentro de la tienda:

```
export interface Product {
  id: number;
  nombre: string;
  categoria: string;
```

```
  marca: string;
  precio: number;
  stock: number;
  descripcion: string;
  imagenUrl: string;
  disponible: boolean;
}
```

Responsabilidad: Estandariza la estructura de los datos devueltos por MockAPI y consumidos por las tarjetas, tablas y formularios.

B. `user.model.ts`

Modela la entidad de usuario autenticado y el contrato de credenciales:

```
export interface User {
  id: number;
  nombre: string;
  email: string;
  rol: string; // 'admin' | 'usuario'
}

export interface LoginCredentials {
  email: string;
  password: string;
}
```

C. `compra.model.ts`

Modela el comprobante de una orden de compra persistida en el endpoint `/compras`:

```
export interface Compra {
  id?: string;
  usuarioEmail: string;
  usuarioNombre: string;
  productoId: number;
  productoNombre: string;
  cantidad: number;
  precioUnitario: number;
  total: number;
  fecha: string; // ISO String para interoperabilidad
}
```

3.2 Capa de Servicios (`src/app/services/`)

A. `AuthService` **(**`auth.service.ts`**)**

- **Responsabilidad:** Gestiona la autenticación de usuarios, la persistencia de sesión en `localStorage`, la difusión del estado del usuario mediante `BehaviorSubject` y el control de roles (`admin` vs `usuario`).
- **Métodos Clave:**
  - `login(credenciales)`: Retorna un `Observable<User>`, validando credenciales contra una nómina controlada y simulando una latencia de red de 600 ms.
  - `logout()`: Elimina la clave del almacenamiento local, emite `null` en el stream reactivo y redirige al `/login`.
  - `isAuthenticated()` y `esAdmin()`: Métodos síncronos de utilidad para los Guards y directivas de plantilla.

B. `ProductService` (`product.service.ts`)

- **Responsabilidad:** Servicio CRUD maestro. Integra modo híbrido: si la variable `environment.apiUrl` está configurada, realiza peticiones HTTP directas a MockAPI; de lo contrario, conmuta automáticamente a una base en memoria con operadores `delay(400)` para garantizar la continuidad del desarrollo.
- **Métodos Clave:**
  - `getProducts()` / `getProductById(id)`: Consultas de colección y detalle.
  - `createProduct(producto)`: Envío `POST` para registrar un nuevo ítem.
  - `updateProduct(id, cambios)`: Envío `PUT` con actualización parcial (`Partial<Product>`).
  - `deleteProduct(id)`: Envío `DELETE`, transformando la respuesta vacía con `map(() => true)`.
  - `comprarProducto(id, cantidad)` y `agregarStock(id, cantidad)`: Algoritmos con validaciones de negocio en el frontend previas a la persistencia.

C. `CompraService` (`compra.service.ts`)

- **Responsabilidad:** Interfaz con el recurso `/compras` de MockAPI.
- **Métodos Clave:**
  - `registrarCompra(compra)`: Realiza una petición `POST` recibiendo los datos inmutables del usuario desde `AuthService`.
  - `getComprasUsuario(email)`: Ejecuta una consulta parametrizada (`GET /compras?usuarioEmail=xxx`), aprovechando el motor de filtrado por URL de MockAPI.
  - `getTodasLasCompras()`: Consulta no filtrada exclusiva para el módulo de auditoría del Administrador.

D. `FoodApiService` (`food-api.service.ts`)

- **Responsabilidad:** Conexión con la API REST pública de *Open Food Facts* (`world.openfoodfacts.org`). Demuestra el principio de Adaptador: recibe un DTO externo no estructurado y lo convierte a una interfaz interna fuertemente tipada:

```
buscarAlimentos(termino: string): Observable<AlimentoExterno[]> {
  const params = {
    search_terms: termino,
    action: 'process',
    json: '1',
    page_size: '12',
```

```
      fields: 'product_name,brands,categories,image_url,nutriscore_grade,nutriments'
    };
    return this.http.get<RespuestaOpenFoodFacts>(this.API_URL, { params }).pipe(
      map(res => this.transformarResultados(res.products)),
      catchError(this.manejarError)
    );
  }
```

3.3 Capa de Seguridad y Protección de Rutas (`src/app/guards/`)

A. `AuthGuard` (`auth.guard.ts`)

Implementa la interfaz `CanActivate` de Angular Router. Evalúa si el usuario posee un token/sesión activo en `AuthService`. En caso negativo, intercepta la navegación y redirige forzosamente hacia `/login`.

B. `RoleGuard` (`role.guard.ts`)

Implementa control de acceso basado en roles (RBAC). Lee la metadata adjunta en la configuración de la ruta (`route.data['roles']`). Si el rol actual del usuario no coincide con la lista permitida, cancela la navegación y redirige al `/inicio`:

```
canActivate(route: ActivatedRouteSnapshot): boolean {
  const rolesPermitidos: string[] = route.data['roles'];
  const rolActual = this.authService.getRole();
  if (rolActual && rolesPermitidos.includes(rolActual)) {
    return true;
  }
  this.router.navigate(['/inicio']);
  return false;
}
```

3.4 Componentes Compartidos (`src/app/shared/`)

A. `ProductCardComponent` (`product-card.component.ts`)

Componente atómico que demuestra la comunicación padre-hijo canónica de Angular:

- **Property Binding con `@Input()`:** Recibe el objeto `producto!: Product` desde el componente contenedor (ej. `DashboardComponent`).
- **Event Binding con `@Output()`:** Emite un evento `EventEmitter<Product>` al invocar el método `onVerMas()`:

```
@Input() producto!: Product;
@Output() verMas = new EventEmitter<Product>();

onVerMas(): void {
```

```
  this.verMas.emit(this.producto);
}
```

- **En el template HTML (`product-card.component.html`):**
  - Interpolación: `{{ producto.nombre }}`
  - Property Binding: `[src]="producto.imagenUrl"` y `[alt]="producto.nombre"`
  - Directivas estructurales: `*ngIf="producto.imagenUrl"`
  - Pipes: `{{ producto.precio | currency:'ARS':'symbol':'1.0-0':'es-AR' }}`

B. `NavbarComponent` (`navbar.component.ts`)

Renderiza la barra de navegación responsive. Se suscribe al `Observable authService.usuario$` en `ngOnInit` para actualizar dinámicamente el nombre y el rol en la barra superior.

- En `app.component.ts`, se controla la visibilidad global del navbar escuchando los eventos de enrutamiento (`router.events`), ocultándolo de manera automática cuando la URL activa es `/login`.

3.5 Vistas Principales y Formularios (`src/app/pages/`)

A. `LoginComponent` (`login.component.ts`)

- **Implementación de Formularios Reactivos:** Utiliza `FormBuilder` para construir un grupo de controles con validadores síncronos:

```
this.formulario = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(6)]]
});
```

- **Control de Feedback:** Métodos accesores (`get email()`, `get password()`) simplifican la lectura del estado de validación en la plantilla (`email.invalid && email.touched`).
- **Angular Material Integration:** Emplea `<mat-spinner>` de diámetro 20 píxeles integrado en el botón de submit para ofrecer feedback durante la autenticación.
- **Acceso Rápido para Evaluadores:** Permite auto-completar el formulario mediante `patchValue()` con botones rápidos para perfiles `Admin` y `Usuario`.

B. `DashboardComponent` (`dashboard.component.ts`)

- **Responsabilidad:** Pantalla de aterrizaje principal tras la autenticación.
- **Cálculo de Métricas Agregadas:** Procesa en memoria la lista de productos obtenida desde `ProductService` para alimentar cuatro tarjetas estadísticas: Total de productos, Disponibles (`stock > 0`), Sin stock (`stock === 0`) y Alerta de stock bajo (`stock <= 10`).
- **Uso del Componente Hijo:** Renderiza los productos destacados mediante directiva `*ngFor` y vincula los eventos `@Output`:

```
<app-product-card
  *ngFor="let producto of productosDestacados"
  [producto]="producto"
  (verMas)="onVerMas($event)">
</app-product-card>
```

C. `ProductosListaComponent` **(**`productos-lista.component.ts`**)**

- **Responsabilidad:** Vista principal de catálogo y administración de inventario.
- **Buscador Reactivo y Filtro de Categoría:** Combina un cuadro de texto reactivo con un selector tradicional.
- **Angular Material Dialog & SnackBar:** La acción de eliminación no utiliza la ventana emergente nativa `confirm()`, sino un cuadro de diálogo accesible con Angular Material:

```
const dialogRef = this.dialog.open(ConfirmarEliminarComponent, {
  width: '400px',
  data: { nombreProducto: producto.nombre }
});
dialogRef.afterClosed().subscribe((confirmado: boolean) => {
  if (confirmado) {
    this.productService.deleteProduct(producto.id).subscribe(() => {
      this.productos = this.productos.filter(p => p.id !== producto.id);
      this.aplicarFiltros();
      this.snackBar.open(`"${producto.nombre}" eliminado.`, 'Cerrar', { duration:
3000 });
    });
  }
});
```

D. `ProductoDetalleComponent` **(**`producto-detalle.component.ts`**)**

- **Parámetros de Ruta:** Captura el parámetro dinámico mediante `this.route.snapshot.paramMap.get('id')`.
- **Doble Lógica según Rol:**
  - **Rol Usuario:** Selector de unidades a comprar, cálculo de subtotal y apertura del modal `ConfirmarCompraComponent`. Tras la confirmación, se actualiza el stock en `/productos` y se emite el comprobante a `/compras`.
  - **Rol Administrador:** Panel de reposición rápida de inventario (`onAgregarStock`) y botones para redirigir a edición o activar el modal de borrado.

E. `ProductoNuevoComponent` **y** `ProductoEditarComponent`

- Emplean validaciones numéricas estrictas (`Validators.min(1)` para precio y `Validators.min(0)` para stock).
- En `ProductoEditarComponent`, el método `cargarProducto()` utiliza `this.formulario.patchValue(...)` para rellenar los controles tan pronto como el observable HTTP

emite el producto existente.

F. `BuscarAlimentosComponent` (`buscar-alimentos.component.ts`)

- Demuestra el uso de **Two-way Data Binding** mediante `[(ngModel)]="terminoBusqueda"`.
- Consume `FoodApiService` y renderiza una cuadrícula de alimentos que muestra la escala oficial de **Nutri-Score** mediante un método auxiliar `getNutriscoreColor(grado)` que aplica códigos hexadecimales dinámicos (`#21960b` para A, `#e63e11` para E).

G. `MisComprasComponent` y `AdminComprasComponent`

- `MisComprasComponent`: Muestra la lista personal del usuario logueado con formato monetario y fecha local argentina (`dd/MM/yyyy HH:mm`), calculando el monto acumulado gastado.
- `AdminComprasComponent`: Panel de auditoría global con filtros locales multidimensionales por nombre de cliente, correo electrónico y descripción del producto adquirido.

3.6 Resumen de Técnicas de Data-Binding y Directivas

Técnica	Sintaxis	Ejemplo en el Proyecto
Interpolación	`{{ expresion }}`	`{{ producto.nombre }}` en tarjetas y tablas
Property Binding	`[propiedad]="variable"`	`[src]="producto.imagenUrl"`, `[disabled]="cargando"`
Event Binding	`(evento)="metodo()"`	`(click)="onLogin()"`, `(verMas)="onVerMas($event)"`
Two-way Binding	`[(ngModel)]="variable"`	`[(ngModel)]="terminoBusqueda"` en buscador nutricional
Directiva Estructural	`*ngIf`, `*ngFor`	`*ngIf="!cargando"`, `*ngFor="let prod of productos"`
Directiva de Clase	`[class.clase]="boolean"`	`[class.is-invalid]="email.invalid && email.touched"`
Pipes de Transformación	`valor | pipe:args`	`precio | currency:'ARS':'symbol':'1.0-0':'es-AR'`

---

4. GUÍA DE INSTALACIÓN, CONFIGURACIÓN Y DESPLIEGUE

4.1 Requisitos Previos del Sistema

Para compilar y ejecutar el proyecto se requiere el siguiente entorno de desarrollo:

1. **Node.js:** Versión `18.19.0` o superior (se sugiere versión `20.x LTS`).
   - Verificación: `node -v`
2. **NPM:** Gestor de paquetes versión `9.x` o `10.x`.
   - Verificación: `npm -v`

3. **Angular CLI:** Interfaz de línea de comandos versión `17.3.x`.
    - Instalación global opcional: `npm install -g @angular/cli@17.3.17`
    - Verificación: `ng version`

4.2 Instalación Paso a Paso

1. **Clonar o descargar el repositorio:**

```
git clone https://github.com/tu-usuario/vida-natural.git
cd vida-natural
```

2. **Instalación de dependencias:** Ejecutar en la raíz del proyecto para descargar todas las librerías declaradas en el archivo `package.json`:

```
npm install
```

3. **Configuración de Variables de Entorno (`environment.ts`):** Verificar el archivo `src/environments/environment.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'https://6ade64134e88f1bba4a52023.mockapi.io/api/v1'
};
```

> **Nota de Arquitectura:** Si se desea utilizar una instancia propia de MockAPI, solo se debe reemplazar el valor de `apiUrl`. Los servicios consumirán automáticamente los endpoints `/productos` y `/compras`.

4.3 Ejecución en Entorno de Desarrollo

Para iniciar el servidor de desarrollo local con recarga en caliente (*Live Reload*):

```
npm start
# O alternativamente:
ng serve -o
```

- La aplicación se compilará y se abrirá automáticamente en el navegador predeterminado bajo la dirección: `http://localhost:4200`

4.4 Compilación y Construcción para Producción

Para compilar el proyecto optimizado, minificado y con reemplazo de variables de entorno para producción:

```
npm run build
```

- Este comando ejecuta internamente: `ng build`.
- Los artefactos finales de producción se generarán en el directorio `dist/vida-natural/browser`.
- El proceso aplica optimizaciones avanzadas: *Tree Shaking*, minificación de JavaScript y CSS, y eliminación de comentarios y código de depuración.

4.5 Análisis de Scripts en `package.json`

```
"scripts": {
  "ng": "ng",
  "start": "ng serve",
  "build": "ng build",
  "watch": "ng build --watch --configuration development",
  "test": "ng test"
}
```

- **`npm start`:** Lanza el servidor web local con soporte para inspección y reconstrucción automática ante modificaciones en los archivos.
- **`npm run build`:** Empaqueta la aplicación completa para su entrega y despliegue en servidores web como Nginx, Apache, Netlify o Vercel.
- **`npm run watch`:** Compila continuamente los archivos en modo desarrollo, ideal para depuración de bundles.
- **`npm test`:** Lanza el entorno de pruebas unitarias mediante Karma y el navegador headless Chrome.

4.6 Instrucciones de Despliegue en la Nube (Netlify)

El proyecto incluye archivos de configuración listos para despliegue continuo:

1. **`netlify.toml`:**

   ```
   [build]
     publish = "dist/vida-natural/browser"
     command = "npm run build"
   ```

2. **`src/_redirects`:**

   ```
   /*    /index.html   200
   ```

Esta regla es **fundamental en aplicaciones Single Page Application (SPA)**: redirige todas las peticiones al archivo `index.html` para que el enrutador de Angular gestione la navegación sin que el servidor responda con un error HTTP 404 al recargar páginas internas como `/productos` o `/inicio`.

Fin del Informe Técnico Académico — Proyecto Vida Natural